CODING PRACTICE – ADVANCED ML

Coding Question 1: DBSCAN – Noise Detection & Cluster Count

Problem Statement

You are given standardized feature data representing user activity on an app.
Your task is to apply **DBSCAN clustering** and determine:

1. The cluster label for each data point
2. The **number of valid clusters formed**, excluding noise points

Noise points are labeled as `-1`.

Input

- `X`: 2D list of standardized values
- `eps`: float
- `min_samples`: integer

Sample Input

```
X = [[0.2, 1.1], [0.3, 1.0], [5.1, 5.0], [5.2, 5.1], [10.0, 10.0]]
eps = 0.5
min_samples = 2
```

Expected Output

```
labels = [0, 0, 1, 1, -1]
cluster_count = 2
```

Solution

```
from sklearn.cluster import DBSCAN

def dbscan_analysis(X, eps, min_samples):
    model = DBSCAN(eps=eps, min_samples=min_samples)
    labels = model.fit_predict(X)

    clusters = set(labels)
    clusters.discard(-1)   # remove noise

    return labels.tolist(), len(clusters)
```

Coding Question 2: DBSCAN – Identify Noise Percentage

Problem Statement

After clustering IoT sensor readings using DBSCAN, management wants to know:

> *What percentage of readings were considered anomalous (noise)?*

Input

- `X`: standardized sensor readings
- `eps`, `min_samples`

Output

- Percentage of points labeled as noise (rounded to 2 decimals)

Sample Input

```
X = [[1,1], [1.1,1.1], [0.9,1.0], [8,8]]
eps = 0.3
min_samples = 2
```

Sample Output

```
25.00
```

Solution

```
import numpy as np
from sklearn.cluster import DBSCAN

def noise_percentage(X, eps, min_samples):
    model = DBSCAN(eps=eps, min_samples=min_samples)
    labels = model.fit_predict(X)

    noise_ratio = np.sum(labels == -1) / len(labels)
    return round(noise_ratio * 100, 2)
```

Coding Question 3: Similarity Matrix – Top-N Similar Items

Problem Statement

You are given a **cosine similarity matrix** between products.
Write a function that returns the **top N most similar products** to a given product, excluding itself.

Input

- `item_name`
- `cosine_sim` (2D list or NumPy array)
- `indices` (mapping from item name → index)
- `n` (number of similar items)

Sample Input

```
item = "A"
indices = {"A": 0, "B": 1, "C": 2, "D": 3}
cosine_sim = [
    [1.0, 0.9, 0.3, 0.2],
    [0.9, 1.0, 0.4, 0.1],
    [0.3, 0.4, 1.0, 0.8],
    [0.2, 0.1, 0.8, 1.0]
]
n = 2
```

Sample Output

```
["B", "C"]
```

Solution

```
def top_n_similar(item, cosine_sim, indices, n):
    idx = indices[item]

    scores = list(enumerate(cosine_sim[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    scores = scores[1:n+1]  # exclude itself
    reverse_indices = {v: k for k, v in indices.items()}

    return [reverse_indices[i[0]] for i in scores]
```

Coding Question 4: Similarity Filtering Above Threshold

Problem Statement

Return **all items** whose similarity score with a given item is **above a specified threshold**.

Input

- item
- cosine_sim
- indices
- threshold

Sample Input

```
item = "A"
threshold = 0.5
```

Sample Output

```
["B"]
```

Solution

```
def similarity_above_threshold(item, cosine_sim, indices, threshold):
    idx = indices[item]

    scores = list(enumerate(cosine_sim[idx]))
    reverse_indices = {v: k for k, v in indices.items()}

    return [
        reverse_indices[i[0]]
        for i in scores
        if i[1] > threshold and i[0] != idx
    ]
```

Coding Question 5: Time Series – Monthly Aggregation

Problem Statement

Given daily sales data, compute the **total sales per month** across all years.

Input

```
dates = ["2022-01-01", "2022-01-15", "2022-02-01", "2023-01-10"]
sales = [10, 20, 15, 25]
```

Expected Output

```
month  total_sales
1      55
2      15
```

Solution

```
import pandas as pd

def monthly_sales(dates, sales):
    df = pd.DataFrame({"date": dates, "sales": sales})
    df["date"] = pd.to_datetime(df["date"])
    df["month"] = df["date"].dt.month

    return (
        df.groupby("month")["sales"]
          .sum()
          .reset_index(name="total_sales")
    )
```

Coding Question 6: Time Series – Rolling Feature Engineering

Problem Statement

Create a **rolling 7-day average sales feature** for time-series data.

Input

```
dates = ["2022-01-01","2022-01-02","2022-01-03",
         "2022-01-04","2022-01-05","2022-01-06","2022-01-07"]
sales = [10, 12, 11, 13, 15, 14, 16]
```

Expected Output

```
rolling_avg_7 = [NaN, NaN, NaN, NaN, NaN, NaN, 13.0]
```

Solution

```
def rolling_average(dates, sales):
    df = pd.DataFrame({"date": dates, "sales": sales})
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values("date")

    df["rolling_avg_7"] = df["sales"].rolling(window=7).mean()
    return df
```